



Computer Arithmetic

Session 2 – Adders



Adders

Revision

Generate, Propagate, Kill

- Generate: $C_{out} = 1$ independent of C
 - $G = A.B$
 - Propagate: $C_{out} = C$
 - $P = A \oplus B$
 - Kill: $C_{out} = 0$ independent of C
 - $K = \bar{A}. \bar{B} = \overline{A \vee B}$
 - $S = A \oplus B \oplus C = P \oplus C$
 - $C_{out} = G \vee (P.C)$
 - $C_{out} = G \vee (\bar{K}.C)$
- P is XOR function and it is too slow, so I will use Kill instead of it to maximum speed between C to C_{out} .

A	B	C	G	P	K	C_{out}	S
0	0	0	0	0	1	0	0
		1				0	1
0	1	0	0	1	0	0	1
		1				1	0
1	0	0	0	1	0	0	1
		1				1	0
1	1	0	1	0	0	1	0
		1				1	1

Generate, Propagate, Kill

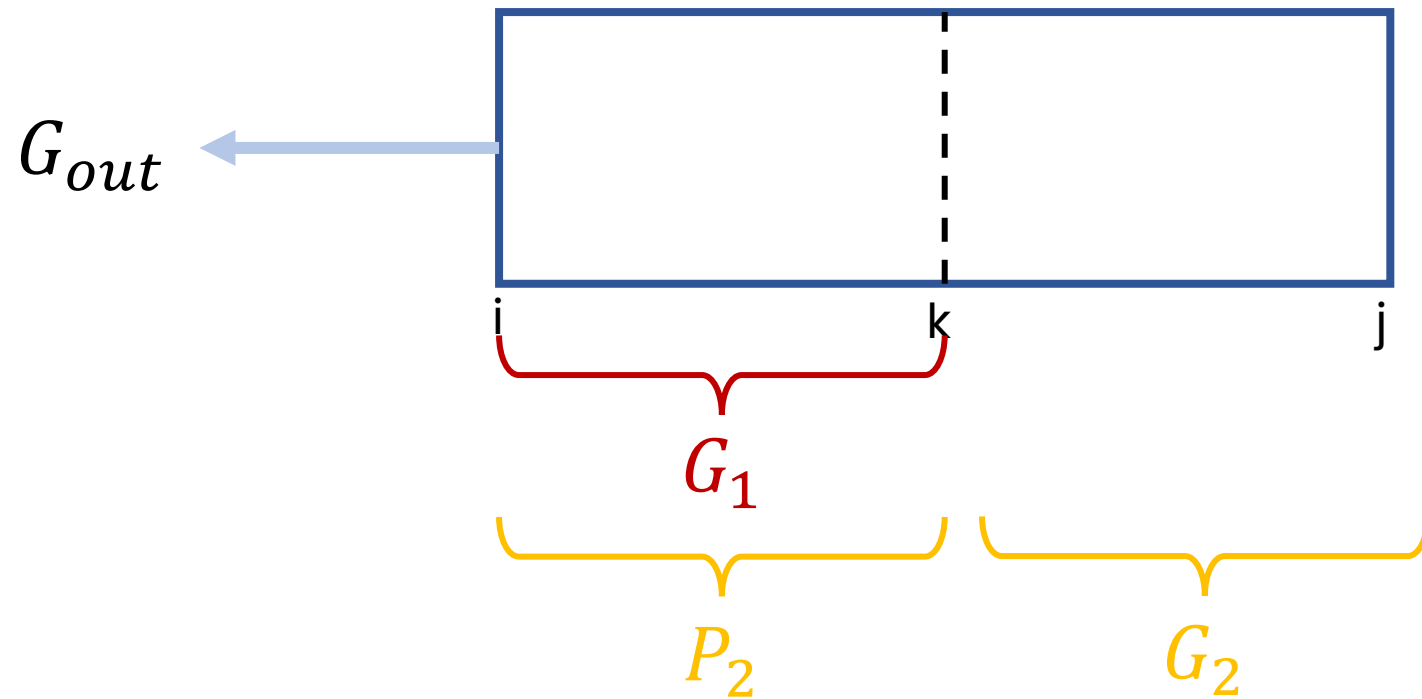


Group PG Signals

- Generalize G and P signals to describe a group of bits $i:j$.
 - A group of bits generates a carry if its carry-out is true independent of the carry-in.
 - A group of bits propagates a carry if its carry-out is true when there is a carry-in.
- Bit-wise generate and propagate signals
 - $G_{i:i} = G_i = A_i \cdot B_i$
 - $P_{i:i} = P_i = A_i \oplus B_i$
- Generate and Propagate for group of bits spanning $i:j$ ($i \geq k > j$)
 - $G_{i:j} = G_{i:k} \vee (P_{i:k} \cdot G_{k-1:j})$
 - $P_{i:j} = P_{i:k} \cdot P_{k-1:j}$

Generate, Propagate, Kill

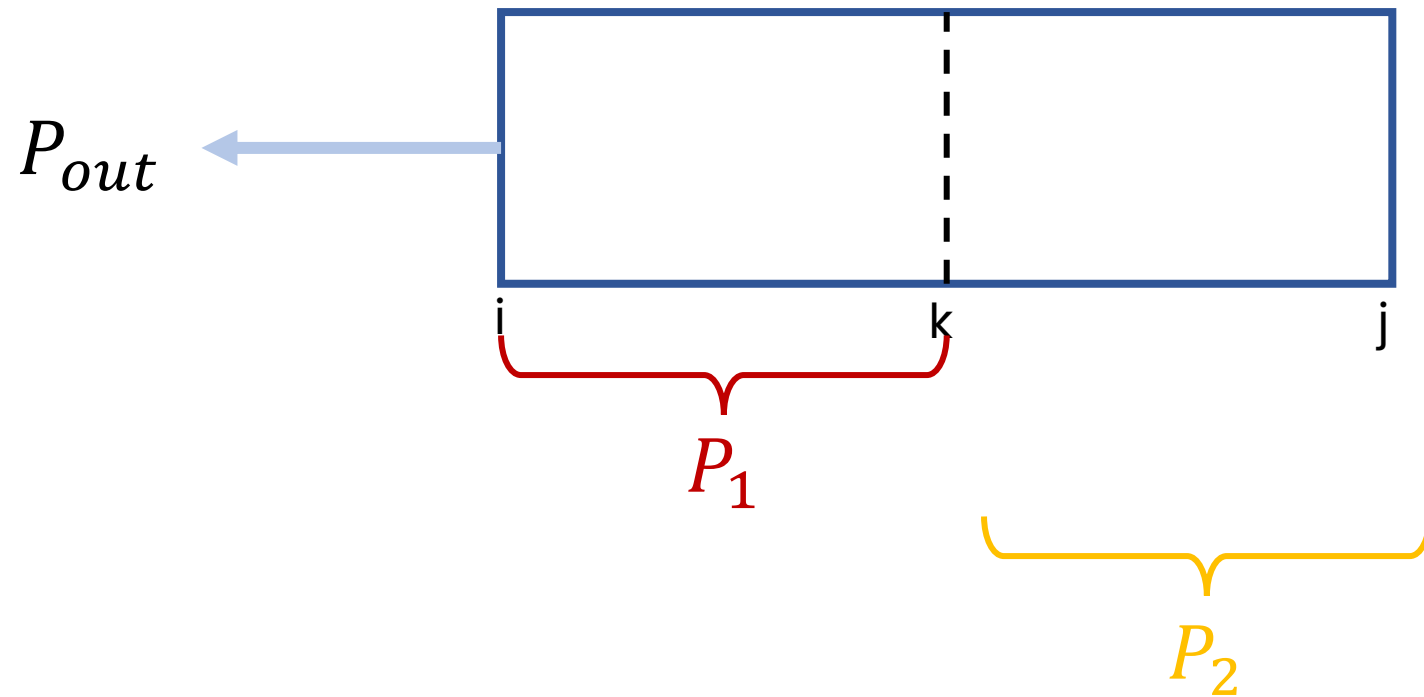
Group PG Signals



$$G_{out} = G_1 \vee G_2 P_2$$

Generate, Propagate, Kill

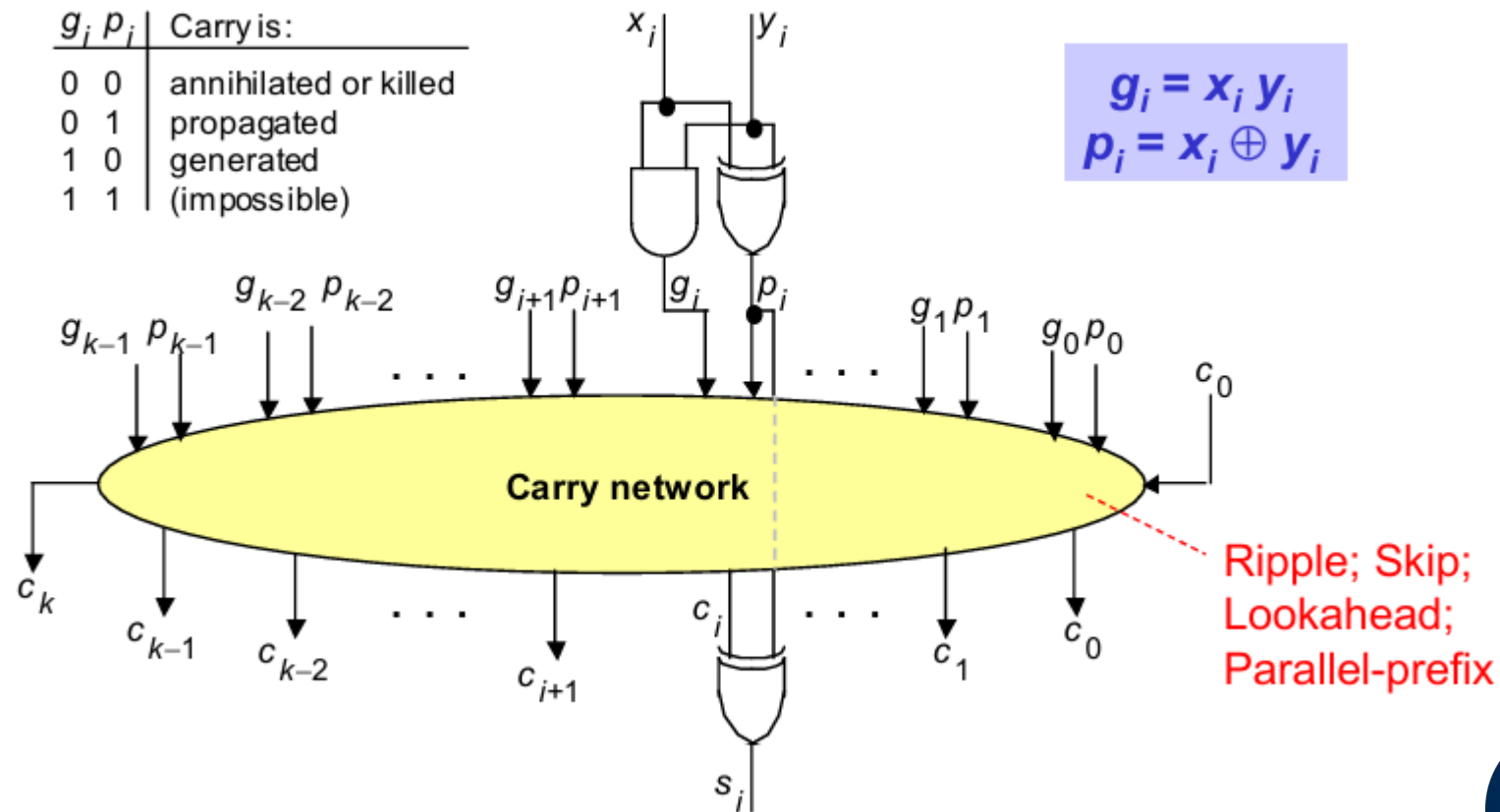
Group PG Signals

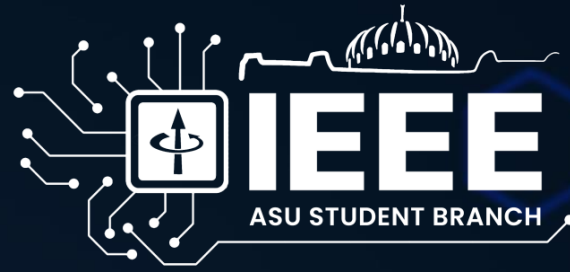


$$P_{out} = P_1 \cdot P_2$$

Carry Network

Carry Network is the Essence of a Fast Adders





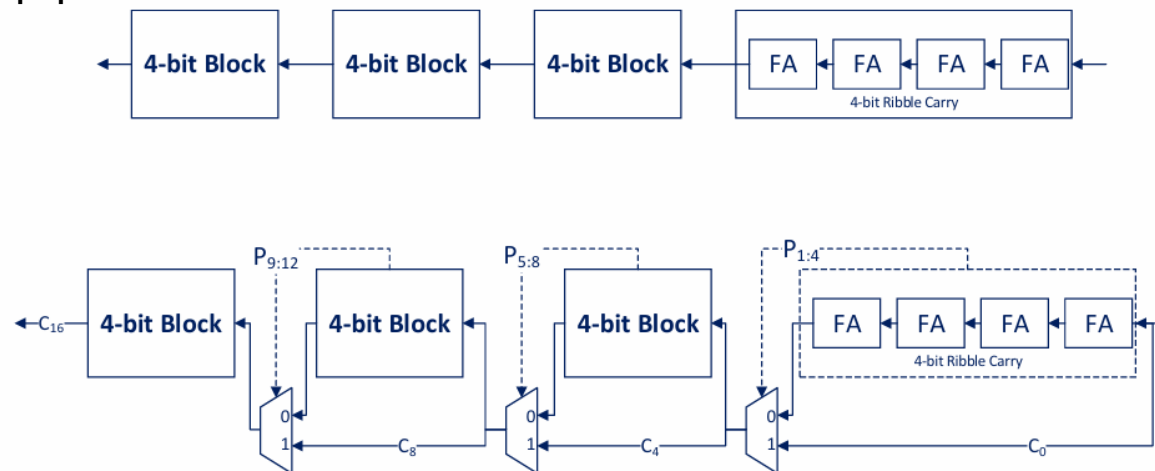
Adders

Fast Adders

Carry Skip Adder

RCA to Carry Skip Adder

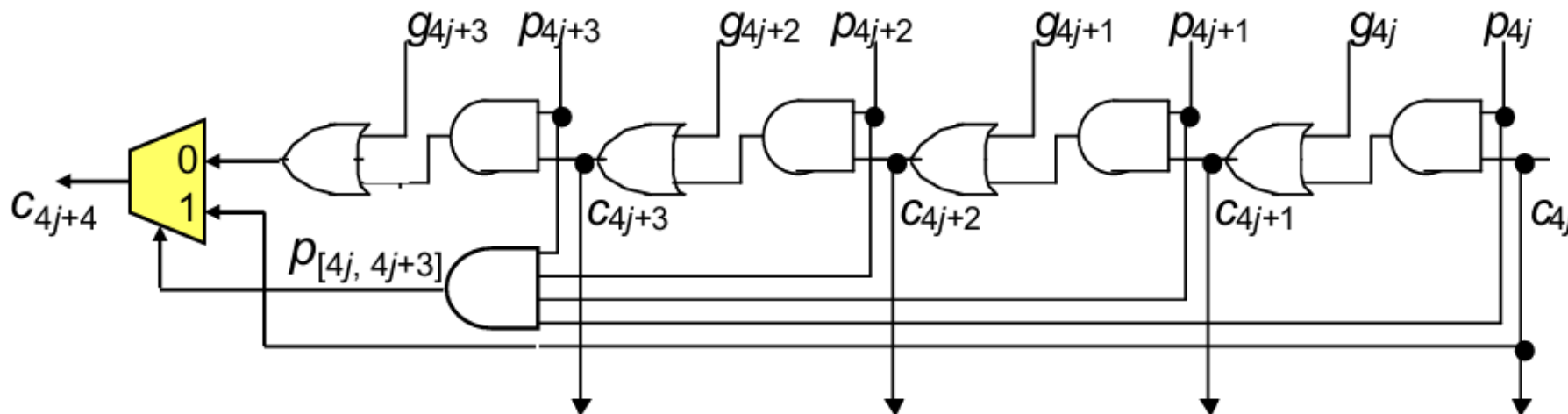
- Consider a 4-bit group or block in a ripple-carry adder, from stage i to stage $i+3$, where i is a multiple of 4. A carry into stage i propagates through this group of 4 bits if and only if it propagates through all four of its stages. Thus, a group propagate signal is defined as $p_{[i,i+3]} = p_i p_{i+1} p_{i+2} p_{i+3}$, which is computable from individual propagate signals by a single four-input AND gate. To speed up carry propagation, one can establish bypass or skip paths around 4-bit blocks.



Carry Skip Adder

RCA to Carry Skip Adder

- Calculated group and propagate each bit (we will use RCA-GP Network)
- Calculate in parallel group Propagate in each stage.
- Calculate 4-bit RCA for every stage, and in parallel 4-bit ripple chains

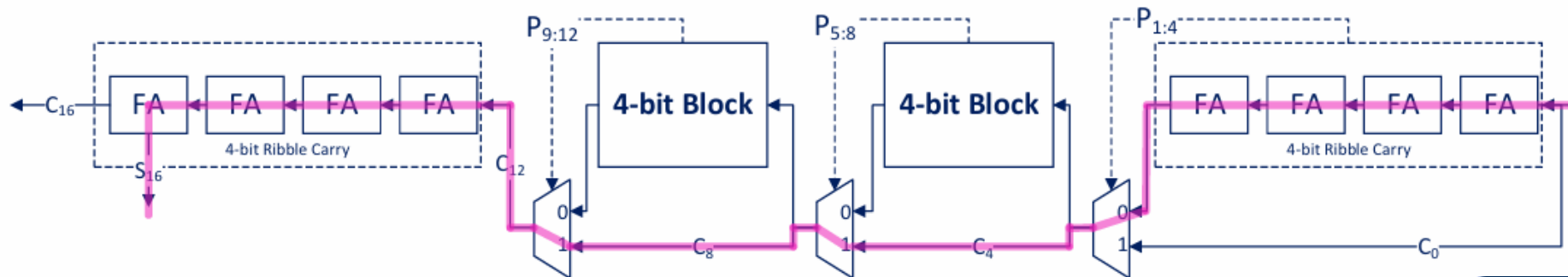


Fast Adders

Carry Skip Adder

Critical Path Through Carry Skip Adder

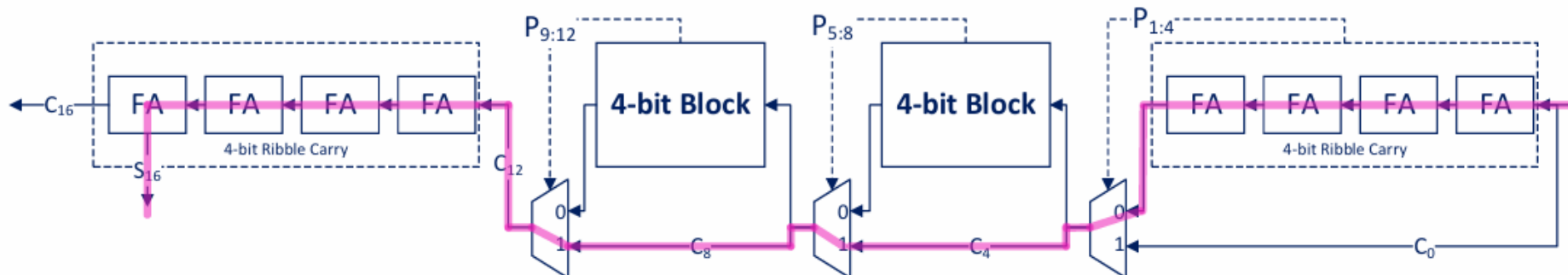
- the worst-case propagation delay, through the carry-skip adder of shown figure corresponds to a carry that is generated in stage 0, ripples through stages 1–3, goes through the multiplexer, skips the middle two groups, and ripples in the last group from stage 12 to stage 15. This leads to 9 stages of propagation (18 gate levels) compared to 16 stages (32 gate levels) for a 16-bit ripple-carry adder.



Carry Skip Adder

Critical Path Through Carry Skip Adder

- Block Width = b , k/b blocks to form a k -bit adder (assume b divides k)
- assume that the delay of the skip multiplexer (mux) is equal to carry propagation delay through one-bit position.
- Delay: $D(k) = (b-1)t_{carry} + \left(\frac{k}{b} - 1\right)t_{mux} + (b-1)t_{carry} + t_{sum} \approx (b-1) + (k/b - 1) + (b-1) \approx 2b + \frac{k}{b} - 3$ stages
- We calculate PG Logic so Canceled t_{sum}

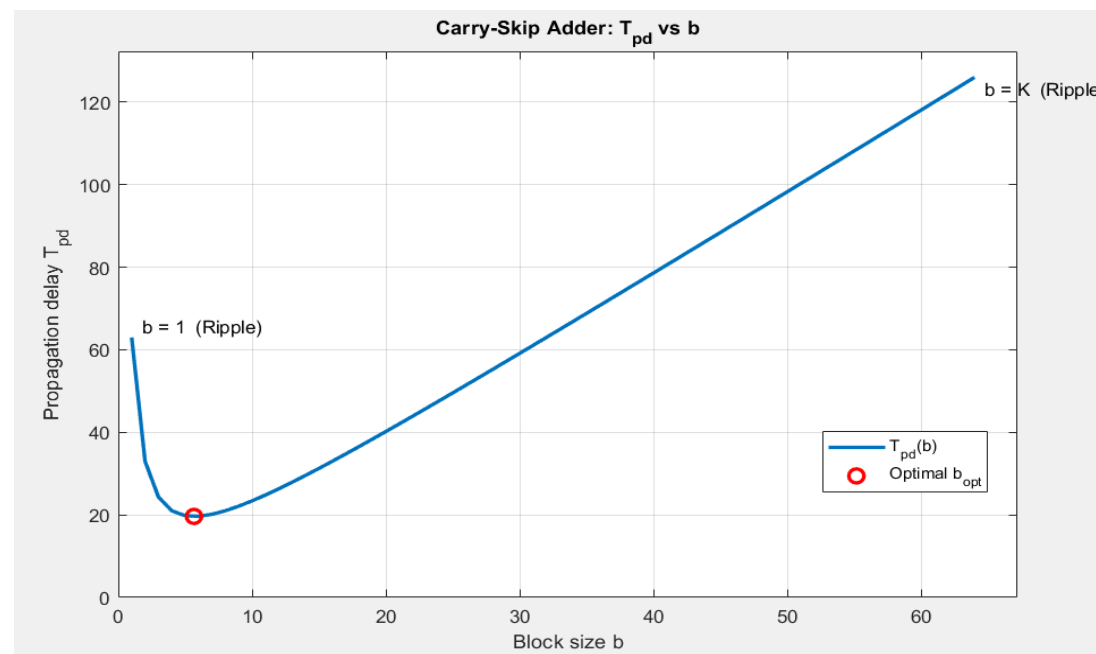
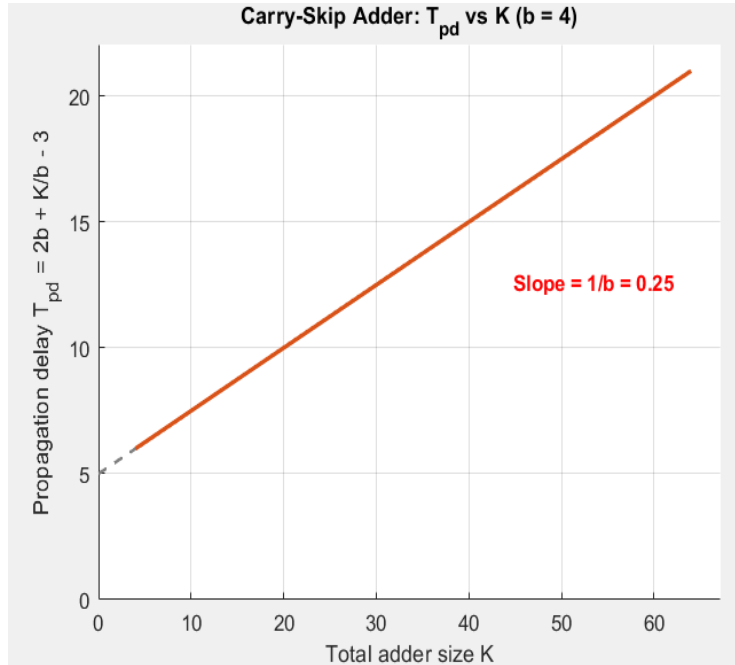
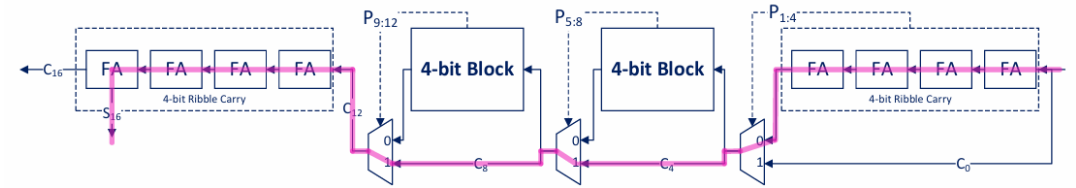


Fast Adders

Carry Skip Adder

Critical Path Through Carry Skip Adder

- Delay: $D(k) = (b - 1) + (k/b - 1) + (b - 1) = 2b + k/b - 3$ stages
- Still Linear Relation as RCA
- How to get $T_{pd_{opt}}$



Carry Skip Adder

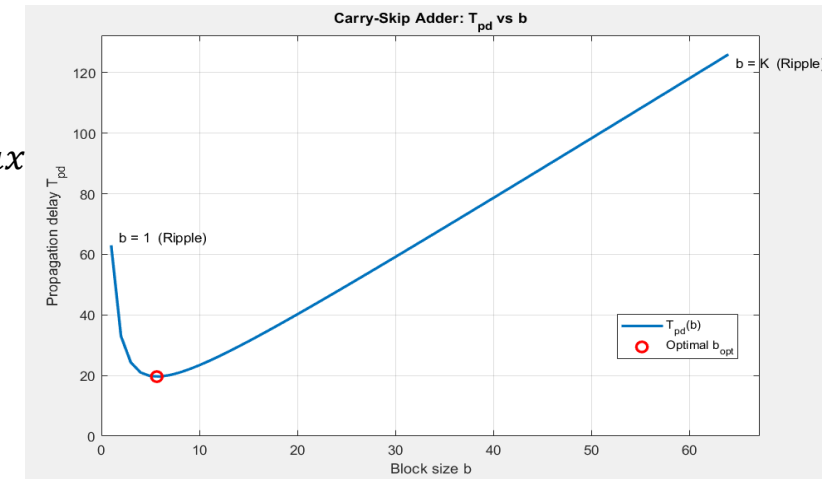
Critical Path Through Carry Skip Adder

- Delay: $D(k) = (b-1)t_{carry} + \left(\frac{k}{b} - 1\right)t_{mux} + (b-1)t_{carry} \approx (b-1) + (k/b - 1) + (b-1) \approx 2b + k/b - 3$

$$T = (b-1)t_{carry} + \left(\frac{k}{b} - 1\right)t_{mux} + (b-1)t_{carry} = 2(b-1)t_{carry} + \left(\frac{k}{b} - 1\right)t_{mux}$$
$$\frac{dT}{db} = 2t_{carry} - \frac{k}{b^2}t_{mux} = 0$$

$$b = \sqrt{\frac{kt_{mux}}{2t_{carry}}} \approx \sqrt{\frac{k}{2}}$$

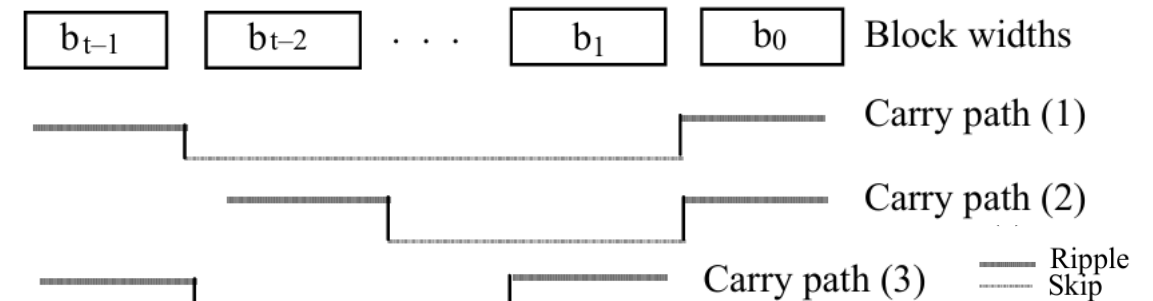
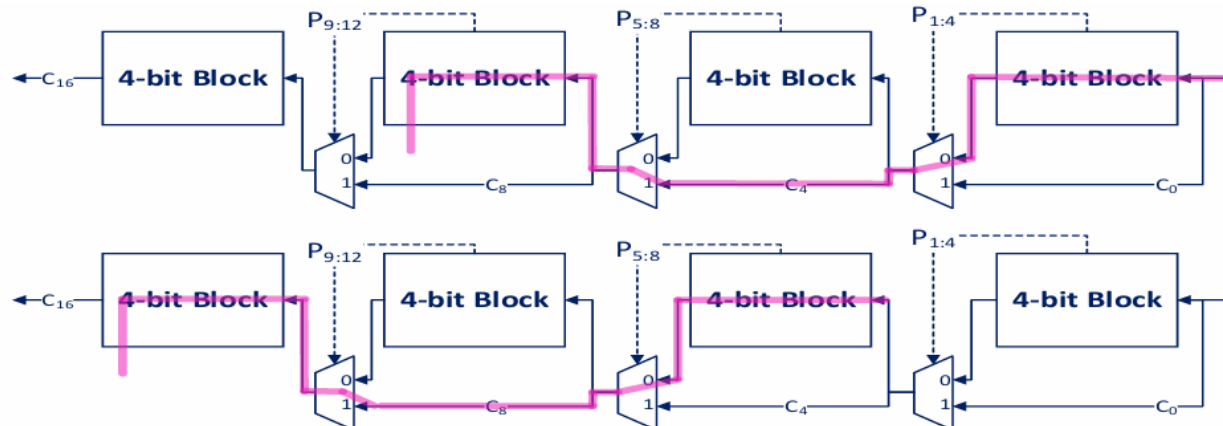
$$T_{opt}(b = \sqrt{\frac{k}{2}}) = 2b + k/b - 3 = 2\sqrt{2k} - 3$$



Carry Skip Adder

Carry Skip Adder With Variable-Size blocks

- Worst-case delay is determined by the **longest carry path**
- Carries originating near the LSB or MSB traverse the largest number of skip stages
- Carries originating in inner blocks traverse fewer skip stages
- Inner blocks can be **wider** without increasing the overall delay



Fast Adders

Carry Skip Adder

Carry Skip Adder With Variable-Size blocks

- Consider three carry paths:

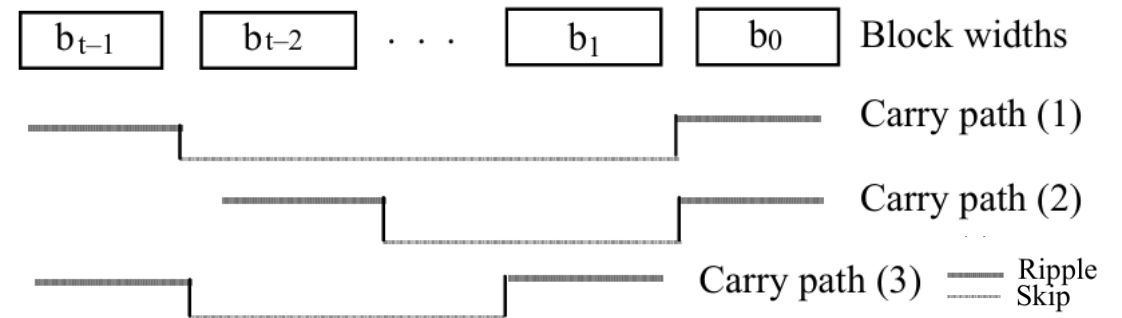
- Path (1): originates in block $t - 1$
- Path (2): originates in block $t - 2$
- Path (3): originates in block 1

- Observations:

- Carry path (2) goes through one fewer skip stage than path (1)
- Carry path (3) goes through **one fewer skip stage** than a carry from block 0

- Conclusion

- Block $t - 2$ can be 1 bit wider than block $t - 1$
- Block 1 can be 1 bit wider than block 0
 - Without increasing total delay.



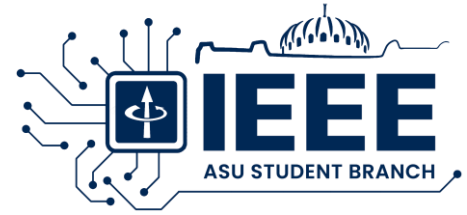
Carry Skip Adder



Optimal Block-Width Pattern

- Assumptions (for ease of analysis):
 - Symmetry: $b_0 = b_{t-1} = b$
 - Even number of blocks: t
- Optimal block widths:
 - $b, b + 1, \dots, b + \frac{t}{2} - 1, b + \frac{t}{2} - 1, \dots, b + 1, b$
 - Small blocks at the edges (critical)
 - Large blocks near the center (non-critical)

Carry Skip Adder



Total Adder Width Constraint

- Total number of bits in the adder (k):
 - $2 \left[b + (b + 1) + \dots + \left(b + \frac{t}{2} - 1 \right) \right]$
- Simplifying:
 - $k = t \left(b + \frac{t}{4} - \frac{1}{2} \right)$
- Solving for the base block size:
 - $b = \frac{k}{t} - \frac{t}{4} + \frac{1}{2}$
- Simplifying happen by using arithmetic series formula

Carry Skip Adder

Delay of Variable Block Carry Skip Adder

- Worst-case delay consists of:
 - Ripple through first block: $b - 1$
 - Ripple through last block: $b - 1$
 - Skip delay through blocks: $t - 2$
 - One skip logic delay

$$T_{var-skip-add} = 2(b - 1) + 1 + (t - 2), \text{ sub } b = \frac{k}{t} - \frac{t}{4} + \frac{1}{2}$$

$$T_{var-skip-add} = \frac{2k}{t} + \frac{t}{2} - 2$$

Carry Skip Adder

Optimal Delay

- Minimize delay with respect to t :

$$\frac{dT}{dt} = -\frac{2k}{t^2} + \frac{1}{2} = 0$$
$$t_{opt} = 2\sqrt{k}$$

- Optimal delay:

$$T_{\text{var-skip-add}}^{\text{opt}} = 2\sqrt{k} - 2$$

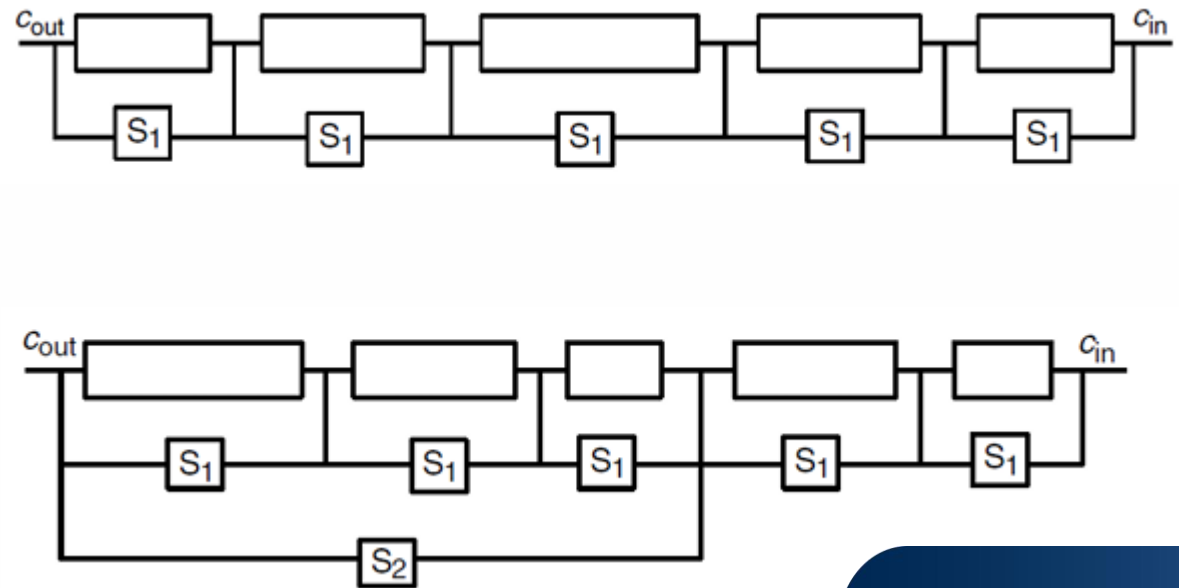
- $T_{\text{fixed-skip-add}}^{\text{opt}} = 2\sqrt{2k} - 3$
- Delay improvement $\approx \sqrt{2}$

Carry Skip Adder

Multilevel Carry-Skip Adder

- Multilevel carry-skip adders are obtained if we allow a carry to skip over several blocks at once.

A two-level carry-skip adder in which second-level skip logic has been provided for the leftmost three blocks. The signal controlling this second-level skip logic is derived as the logical AND of the first-level skip signals. A carry that would need 3-time units to skip these three blocks in a single-level carry-skip adder can now do so in 1 time unit.



Fast Adders

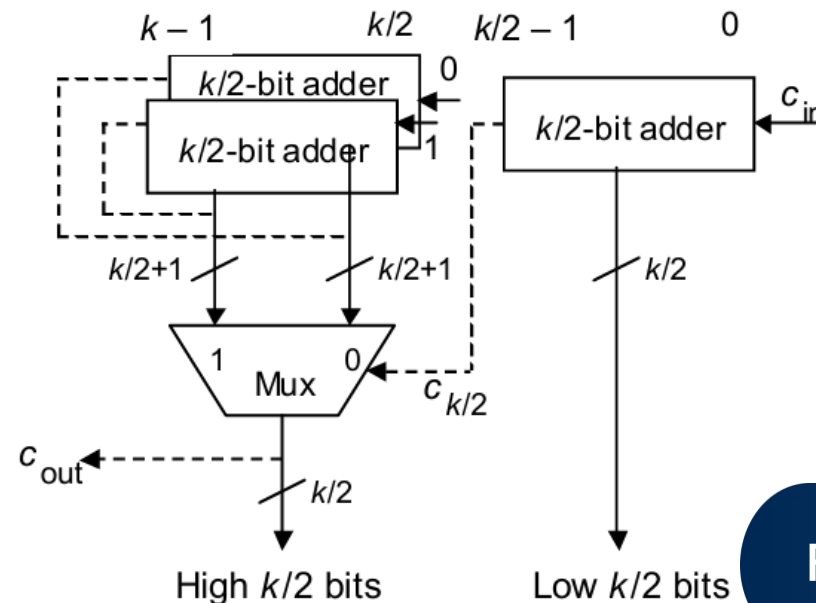
Carry Select Adder

Reducing Carry Propagation Delay

- Compute sum assuming **both possible carry-in values**
- Select the correct result once the true carry becomes known
- Carry propagation is replaced by parallel computation and late selection.

$$C_{\text{select-add}}(k) = 3C_{\text{add}}(k/2) + k/2 + 1$$

$$T_{\text{select-add}}(k) = T_{\text{add}}(k/2) + 1$$



Fast Adders

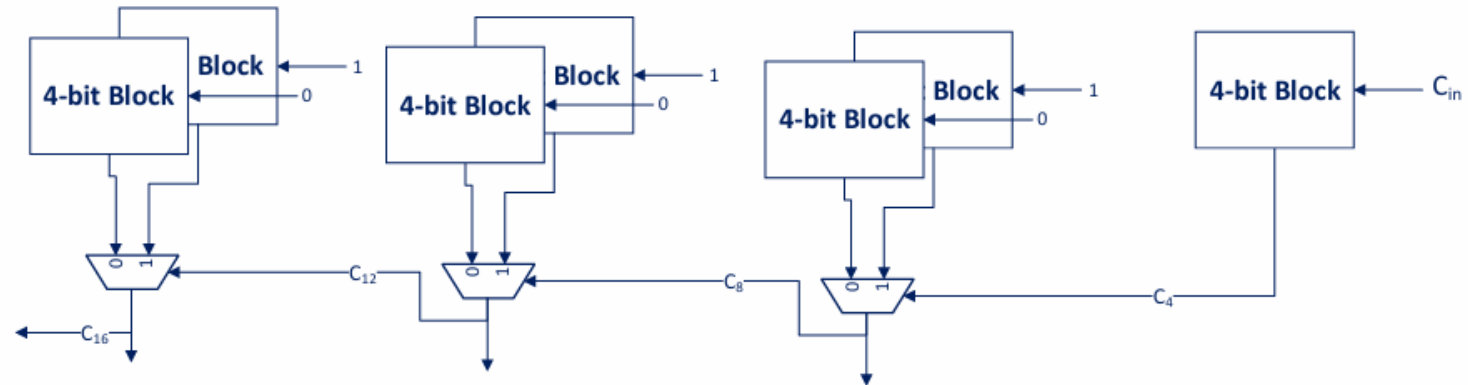
Carry Select Adder

Reducing Carry Propagation Delay

- Compute sum assuming **both possible carry-in values**
- Select the correct result once the true carry becomes known
- Carry propagation is replaced by parallel computation and late selection.

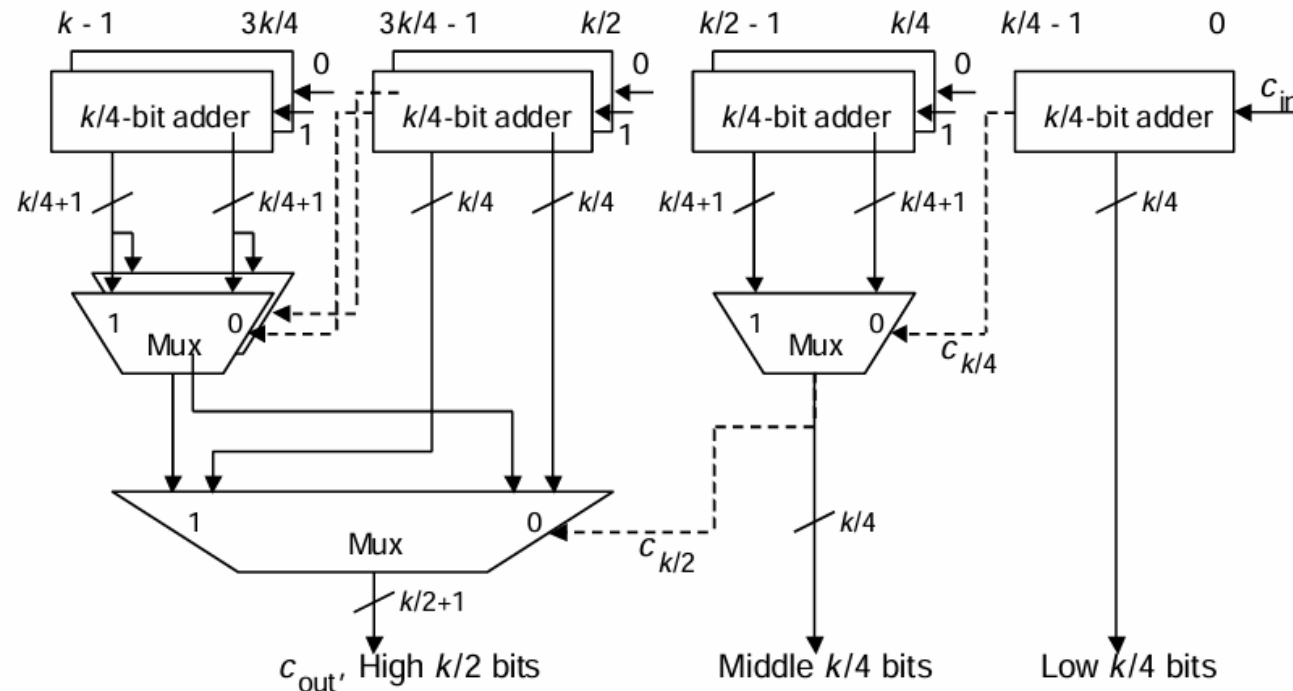
$$T_{\text{select-add}}(k) = T_{\text{add}}(b) + (t-1) * T_{\text{mux}}$$

$$C_{\text{select-add}}(k) = (2t-1) * C_{\text{add}}(b) + \frac{\text{add}}{(t-1) * (b+1)} \frac{\text{mux}}$$



Carry Select Adder

Multilevel Carry Select Adder

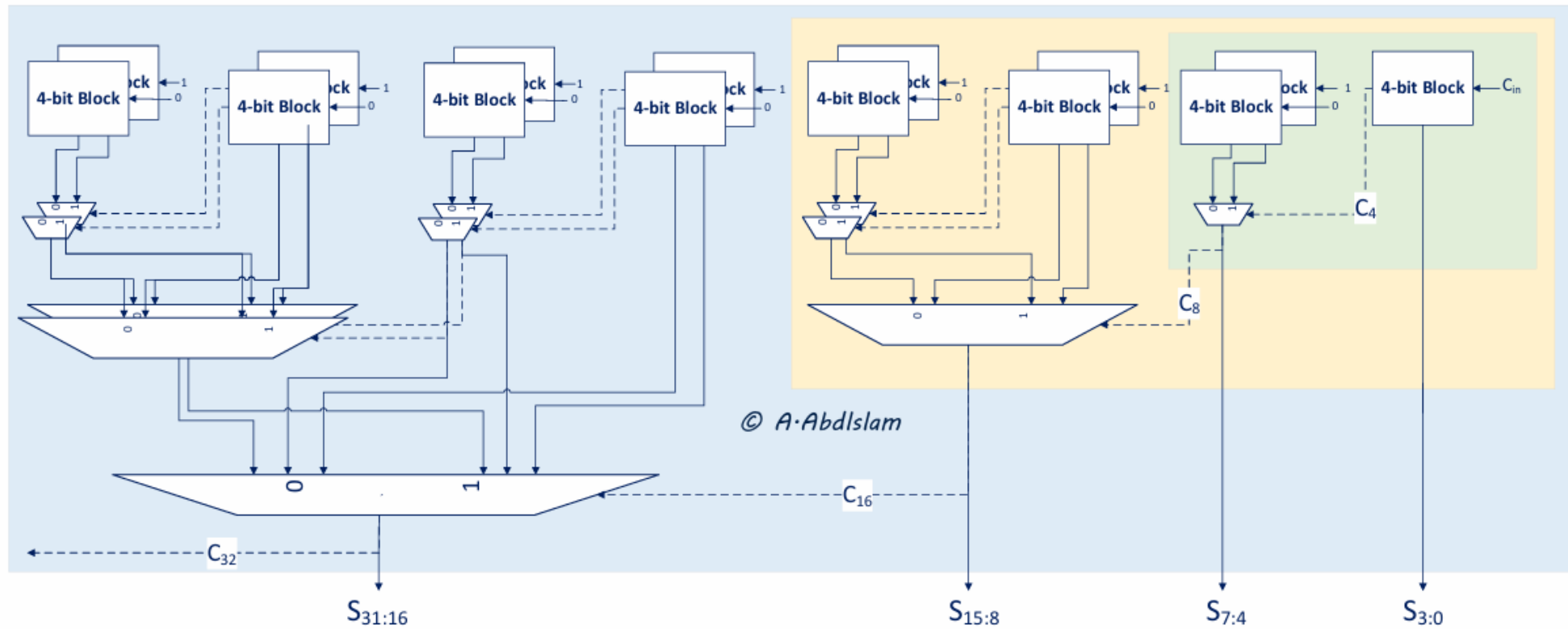


The logo for the IEEE ASU Student Branch. It features the IEEE logo (a stylized 'I' with a vertical line and a horizontal line) inside a square frame, which is connected to a circuit-like pattern of lines and dots. To the right of the square frame is the text 'IEEE' in a large, bold, sans-serif font. Below 'IEEE' is the text 'ASU STUDENT BRANCH' in a smaller, bold, sans-serif font. Above the 'IEEE' text is a stylized dome, representing the ASU building.

Fast Adders

Carry Select Adder

Multilevel Carry Select Adder



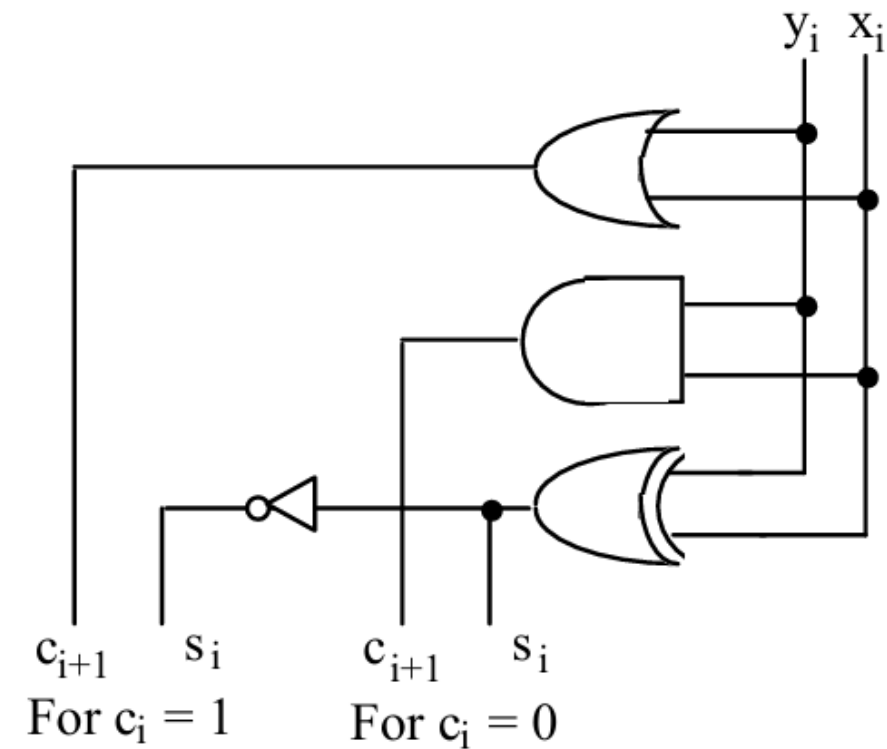
Fast Adders

Conditional Sum Adder

Extreme multilevel carry-select with 1-bit blocks

Block width	Block carry-in		Block sum and block carry-out																C_{in}
			15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
1	0	s	0	1	1	0	1	1	0	1	1	0	1	1	0	1	1	1	0
	1	c	0	0	0	0	0	1	0	1	0	1	0	0	1	0	0	0	0
2	0	s	0	1	1	0	1	1	0	1	0	0	1	1	0	1	1	1	0
	1	c	1	0	0	1	0	0	1	0	0	1	0	0	1	0	0	0	0
4	0	s	0	1	1	0	0	0	0	1	0	0	1	1	0	1	1	1	0
	1	c	1	0	1	1	0	0	1	0	0	1	0	0	1	0	0	0	0
8	0	s	0	1	1	1	0	0	0	1	0	1	0	0	0	1	1	1	0
	1	c	0	1	1	1	0	0	1	0	0	1	0	0	0	0	0	0	0
16	0	s	0	1	1	1	0	0	1	0	0	1	0	0	0	1	1	1	0
	1	c	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

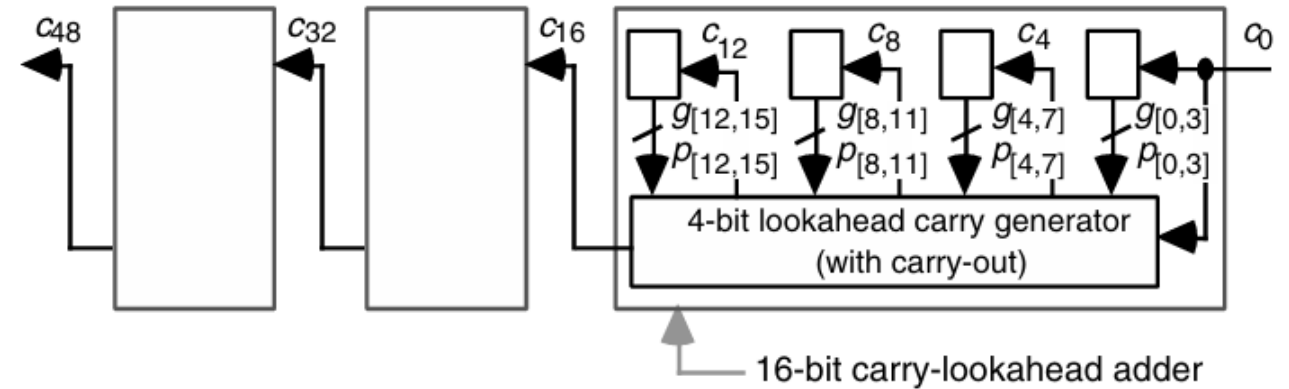
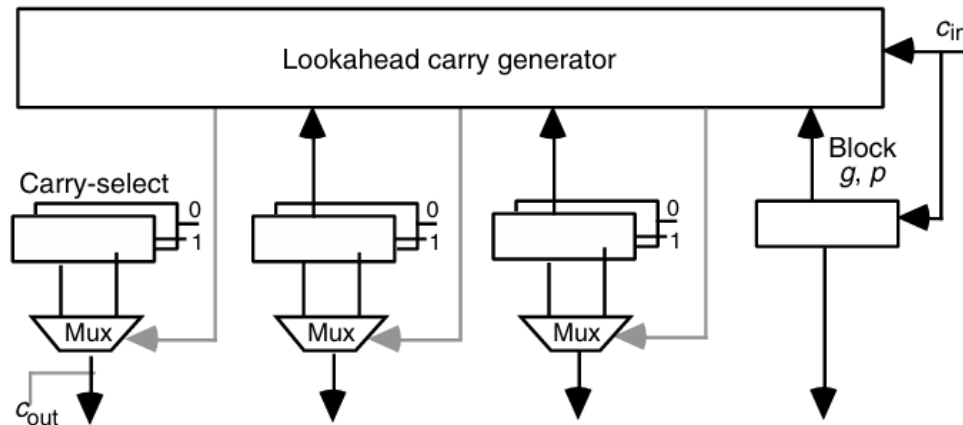
C_{out}



Fast Adders

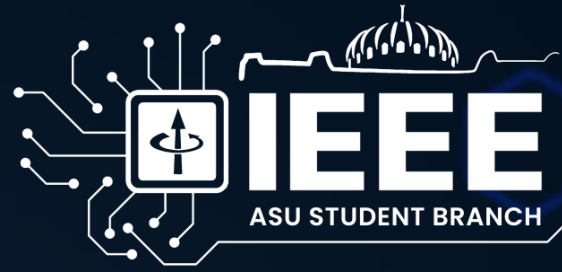
Hybrid Adders

A hybrid Carry-Lookahead/Carry-Select adder



A hybrid Ripple-Carry/Carry-Lookahead

Fast Adders



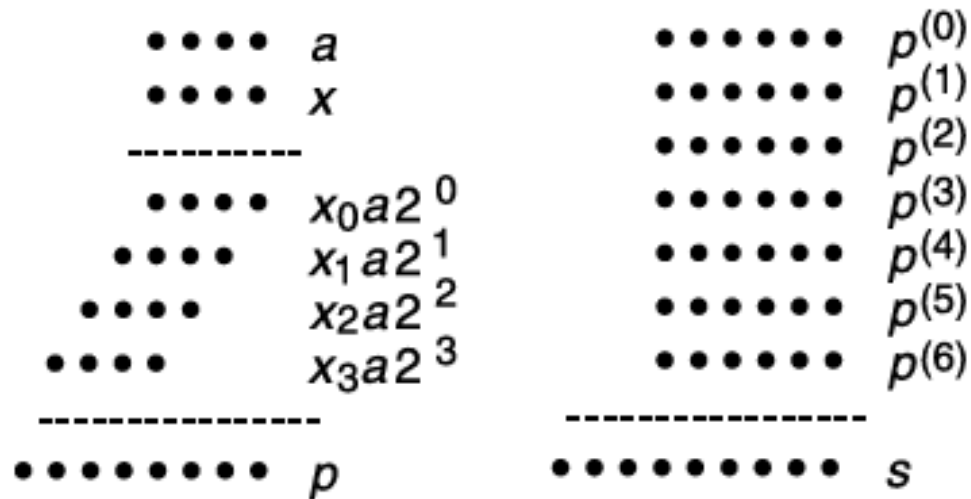
Adders

Multiperand Addition

Multioperand Addition

Multioperand addition is implicit in both multiplication and computation of vector inner products

Multioperand addition problems for multiplication or inner-product computation shown in dot notation.

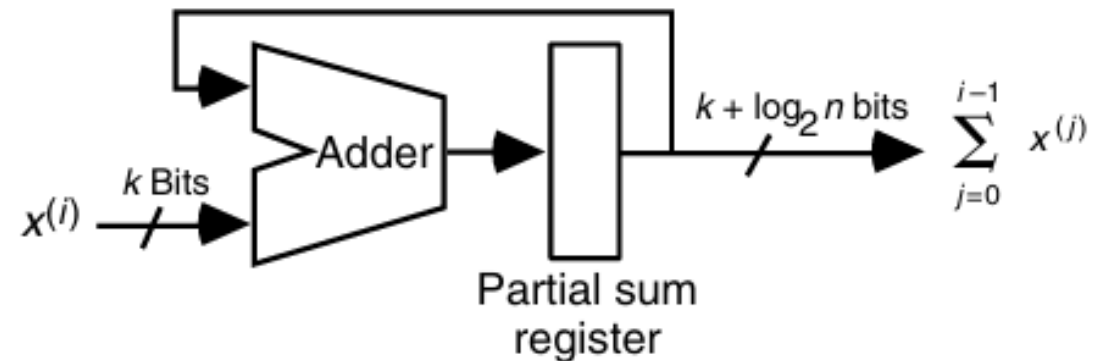


Result Number of bits = $k + \log_2(n)$
k: Number of bits of one operand
n: Number of operands

Using Two Operand Adders

Serial implementation of multioperand addition:

- It depicts a serial solution to the multioperand addition problem using a single two-operand adder.
- Assume we use a fast adder as the adder.
- Therefore, addition time grows super linearly with n when k is fixed and logarithmically with k for a given n .
- Couldn't pipeline straight forward.

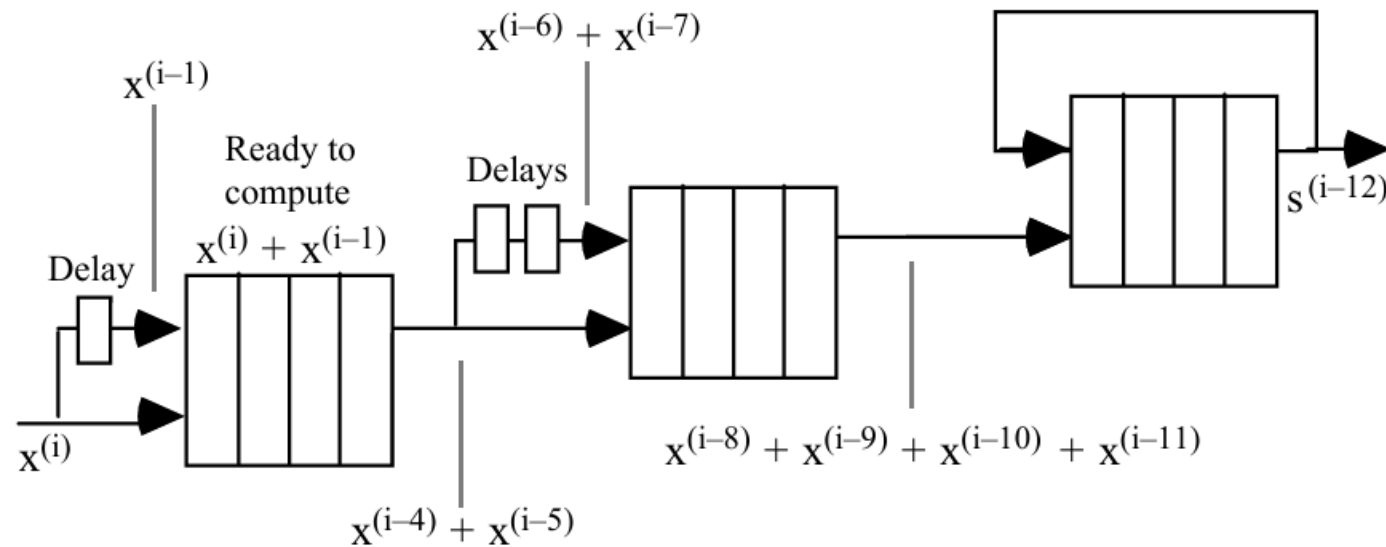


$$\begin{aligned} T_{\text{serial-multi-add}} &= O(n \log(k + \log n)) \\ &= O(n \log k + n \log \log n) \end{aligned}$$

Using Two Operand Adders

Serial implementation of multioperand addition (Pipelined):

- Even though the clock cycle is now shorter because of pipelining, the latency from the first input to the last output remains asymptotically the same with h -stage pipelining for any fixed h .



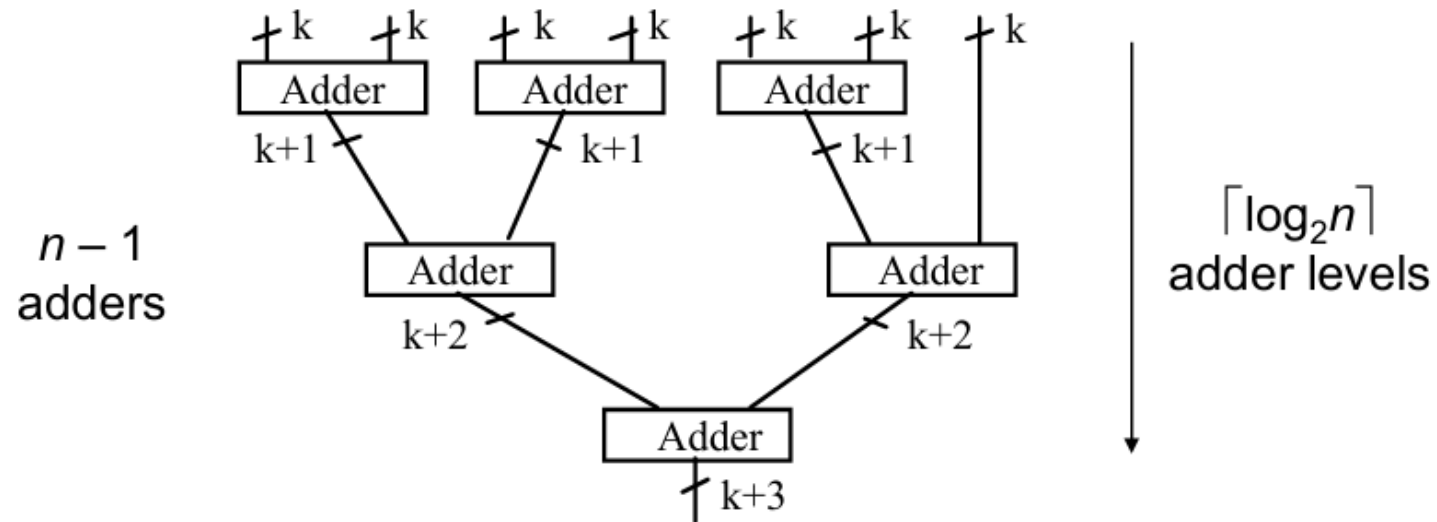
This scheme achieves a Speedup of 4 with 3 Adders

Multioperand Addition

Using Two Operand Adders

Parallel Implementation as Tree of Adders

- Assume we use a fast adder as the adder.



$$T_{\text{tree-fast-multi-add}} = O(\log k + \log(k+1) + \dots + \log(k + \lceil \log_2 n \rceil - 1))$$

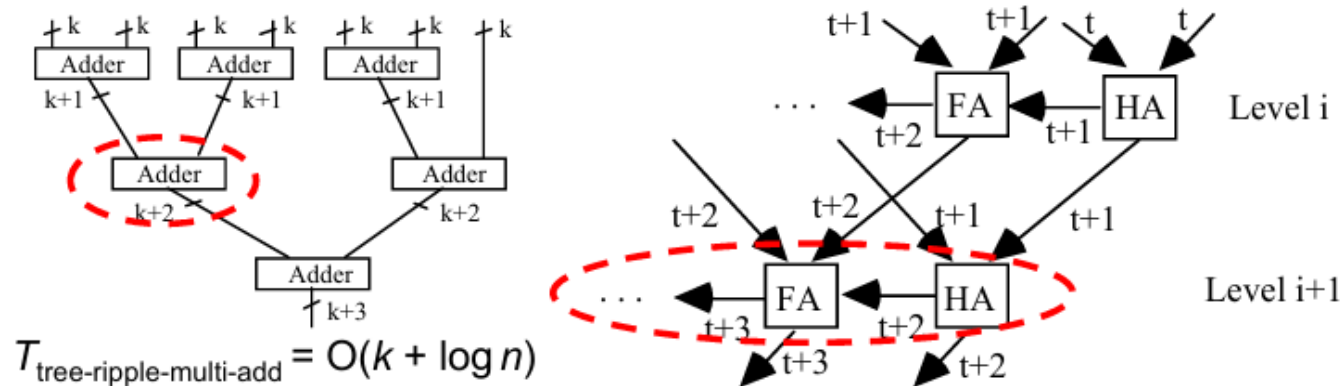
$$= O(\log n \log k + \log n \log \log n)$$

$$T_{\text{tree-ripple-multi-add}} = O(k + \log n)$$

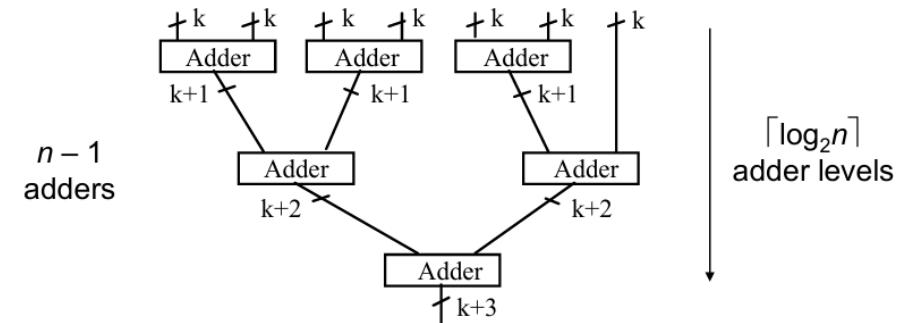
Using Two Operand Adders

Parallel Implementation as Tree of Adders

- Using Ripple Carry Adder as the adder.



Ripple-carry adders at levels i and i + 1 in the tree of adders used for multi-operand addition.



$$T_{\text{tree-ripple-multi-add}} = O(k + \log n)$$

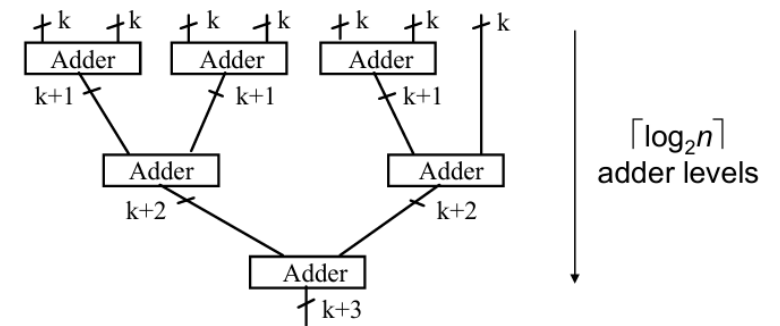
Using Two Operand Adders

Parallel Implementation as Tree of Adders

- The absolute best latency that we can hope for is $O(\log k + \log n)$
- There are kn data bits to process and using any set of computation elements with constant fan-in, this requires $O(\log(kn))$ time
- We will see shortly that carry-save adders achieve this optimum time

$$\begin{aligned} T_{\text{tree-fast-multi-add}} &= O(\log k + \log(k+1) + \dots + \log(k + \lceil \log_2 n \rceil - 1)) \\ &= O(\log n \log k + \log n \log \log n) \end{aligned}$$

$$T_{\text{tree-ripple-multi-add}} = O(k + \log n)$$



Carry-Save Adders

Addition of Redundant Numbers

Carry Propagating Addition

6	2	3	4	5	7
2	2	9	6	4	7
8	5	3	1	0	4

Operand digit in [0:9]

Carry-Free Addition

6	2	3	4	5	7
2	2	9	6	4	7
8	4	12	10	9	14

Operand digit in [0:9]

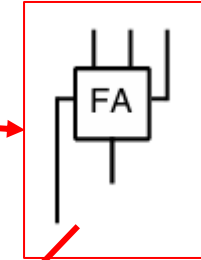
Operand digit in [0:14]

Carry Save Adder (CSA)

Carry Store Adder

3:2 Compressor

3:2 Counter



Carry-Free Addition by Carry store Addition

6	2	3	4	5	7
2	2	9	6	4	7
8	4	2	0	9	4
0	0	1	1	0	1
8	5	3	1	0	4

Operand digit in [0:9]

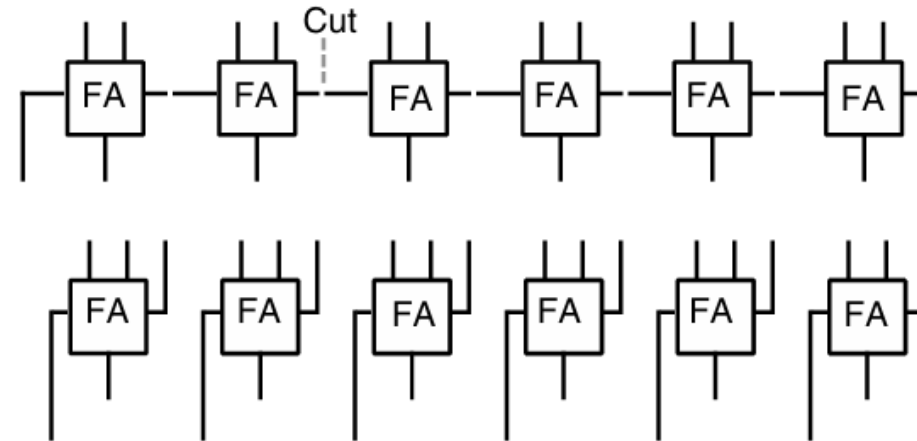
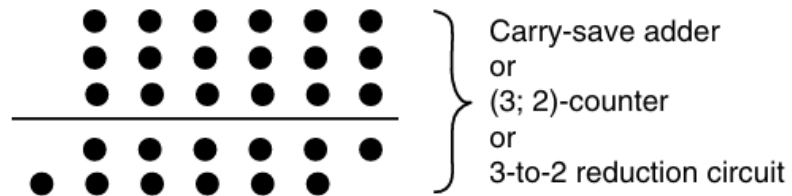
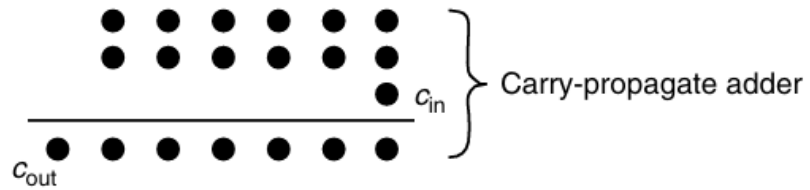
Operand digit in [0:9]

Operand digit in [0:9]

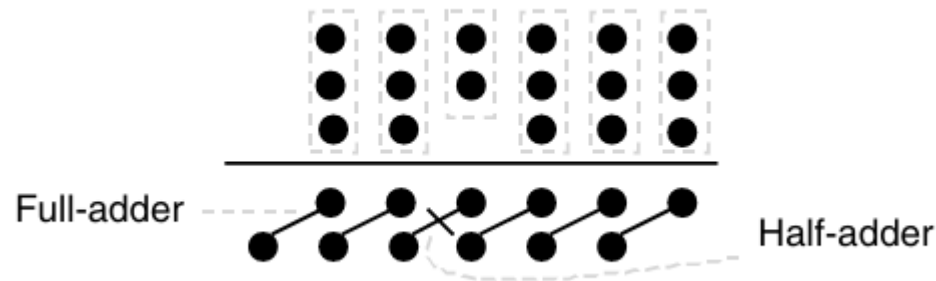
Multioperand Addition

Carry-Save Adders

A Ripple Carry Adder turns into Carry-Save Adder



Using Half Adder:



Multioperand Addition

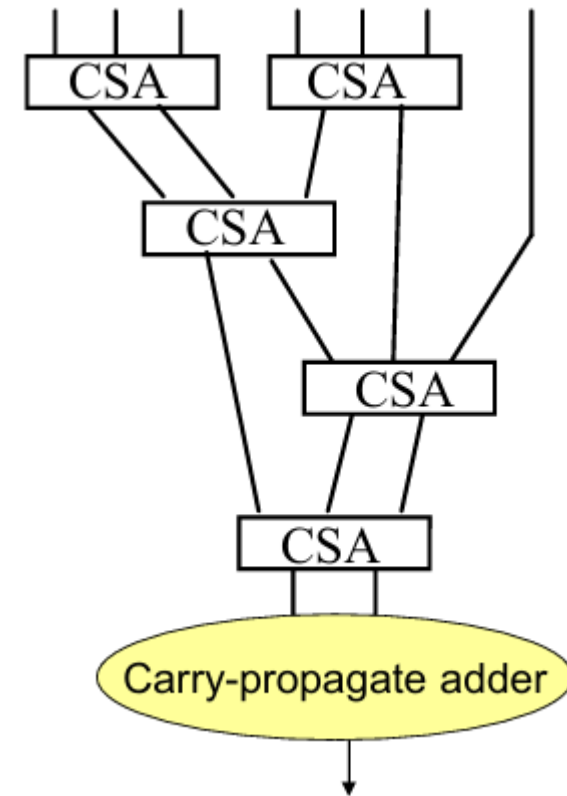
Carry-Save Adders

Multiperand Addition Using CSAs

- Tree of carry-save adders reducing seven numbers to two.
- The needed CSAs are of various widths, but generally the widths are close to k bits; the CPA is of width at most $k + \log_2 n$

$$\begin{aligned}T_{\text{carry-save-multi-add}} &= O(\text{tree height} + T_{\text{CPA}}) \\ &= O(\log n + \log k)\end{aligned}$$

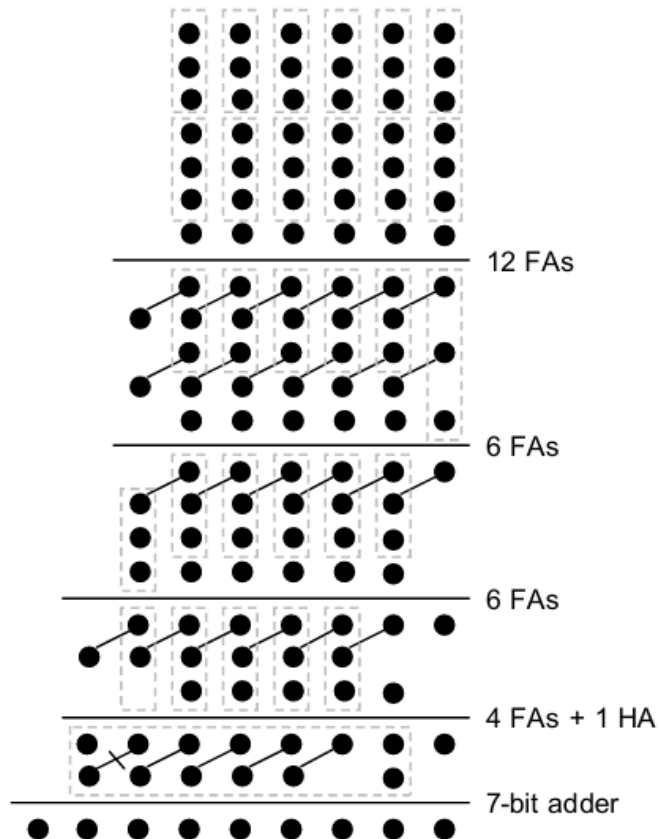
$$C_{\text{carry-save-multi-add}} = (n - 2)C_{\text{CSA}} + C_{\text{CPA}}$$



Multiperand Addition

Carry-Save Adders

Example Reduction by a CSA Tree



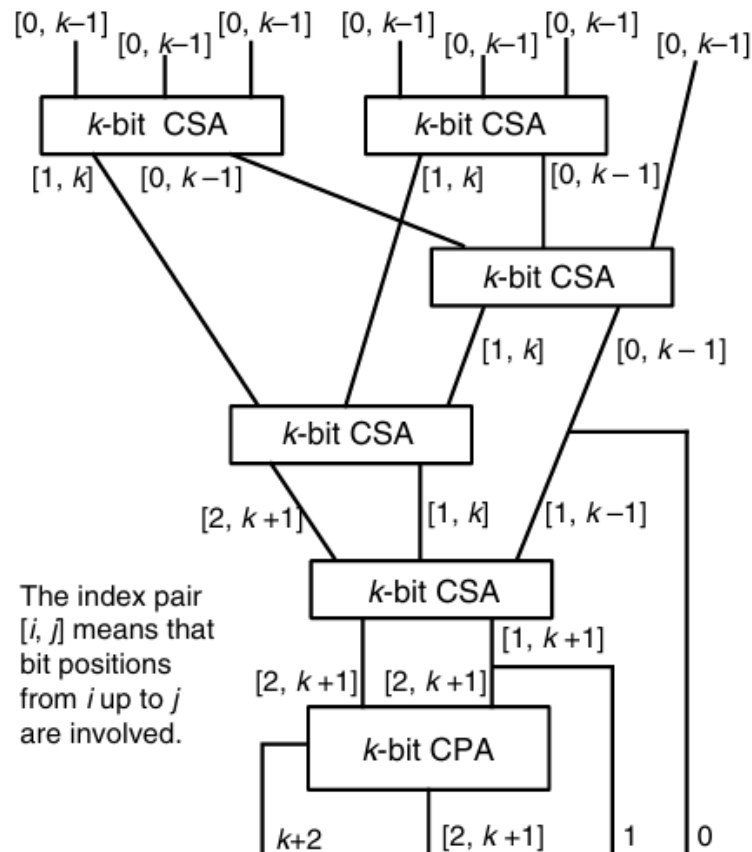
Total cost = 7-bit adder + 28 FAs + 1 HA

8	7	6	5	4	3	2	1	0	Bit position
			7	7	7	7	7	7	6×2 = 12 FAs
		2	5	5	5	5	5	3	6 FAs
		3	4	4	4	4	4	1	6 FAs
	1	2	3	3	3	3	2	1	4 FAs + 1 HA
	2	2	2	2	2	1	2	1	7-bit adder
--Carry-propagate adder--									
1	1	1	1	1	1	1	1	1	

Multioperand Addition

Carry-Save Adders

Width of Adders in a CSA Tree

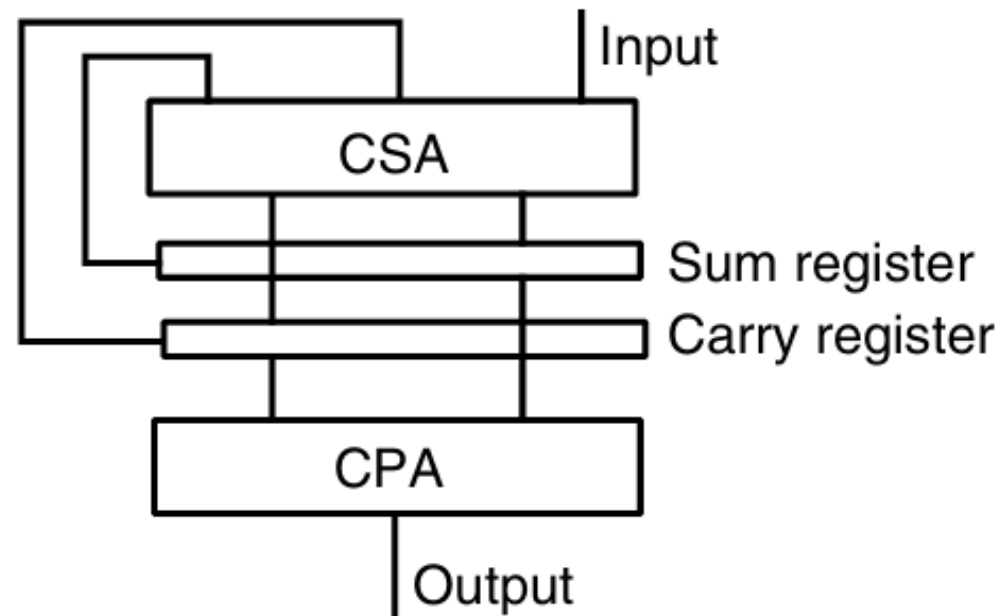


- Adding Seven k -bit numbers and the CSA/CPA Widths required
- Due to the gradual retirement (dropping out) of some of the result bits, CSA widths do not vary much as we go down the tree levels

Carry-Save Adders

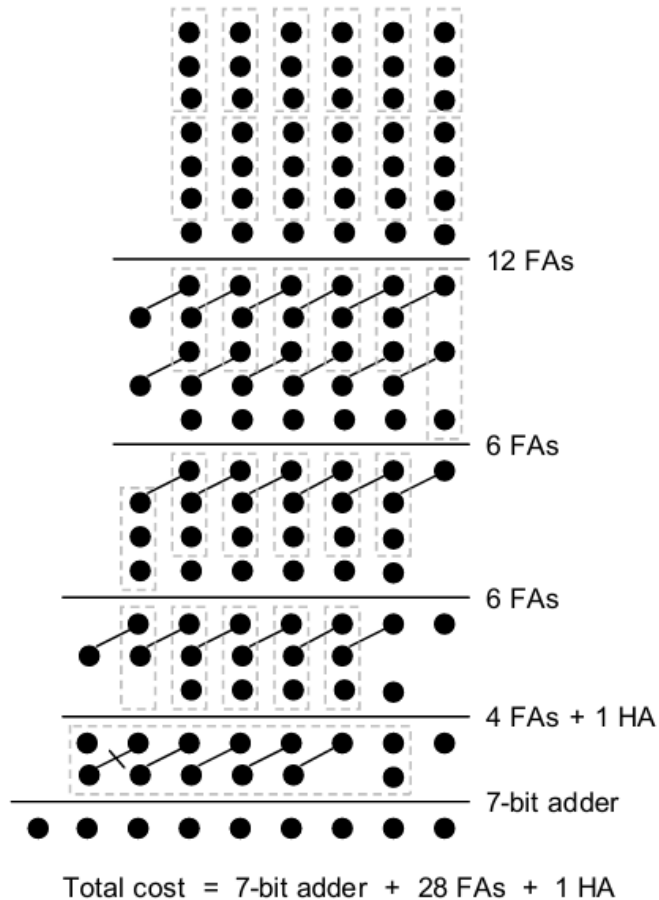
Serial Carry-Save Addition by means of a single CSA

- better implementation for the serial addition technique (smaller cycle delay single FA)



Wallace Trees

Reducing the Number of Operands as Early as possible



$$r_{j+1} = 2 * \text{floor} \left(\frac{r_j}{3} \right) + r_j \bmod(3)$$

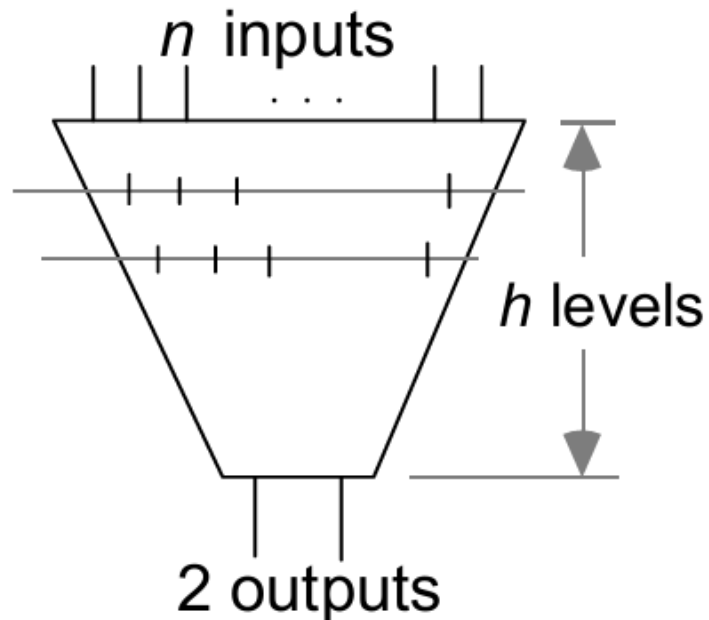
3 Comes from Full Adder is 3:2 Compressor

here in this example: $r_0 = 6 \rightarrow r_1 = 4 \rightarrow r_2 = 3 \rightarrow r_3 = 2$

CSA Reduction Tree

Maximum Number of Operands per Level

- $d_{k+1} = \text{floor}\left(\frac{3}{2} \times d_k\right)$
- $d_0 = 2$
- $d_0 = 2 \rightarrow d_1 = 3 \rightarrow d_2 = 4 \dots$



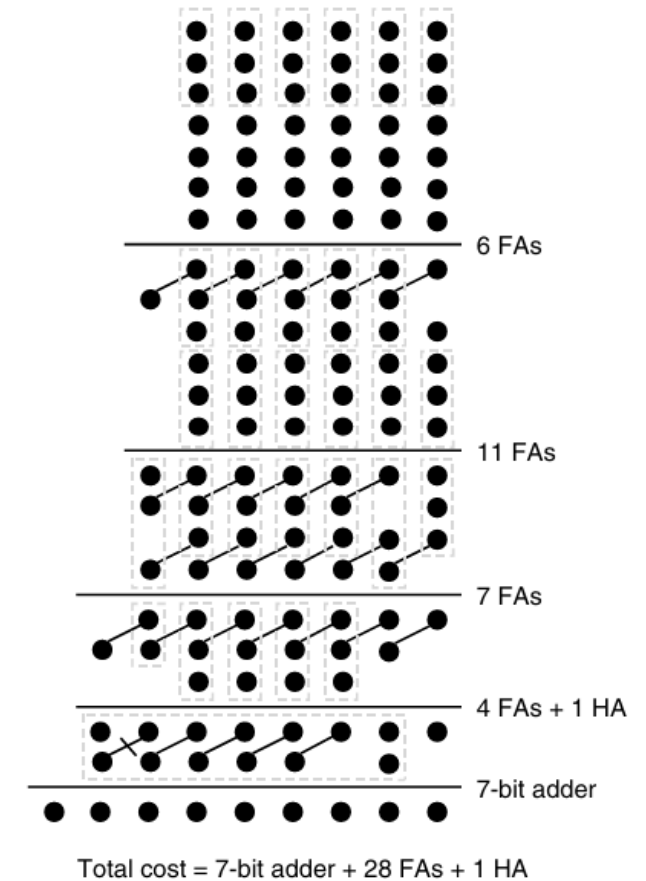
h	$n(h)$	h	$n(h)$	h	$n(h)$
0	2	7	28	14	474
1	3	8	42	15	711
2	4	9	63	16	1066
3	6	10	94	17	1599
4	9	11	141	18	2398
5	13	12	211	19	3597
6	19	13	316	20	5395

$n(h)$: Maximum number of inputs for h levels

Dadda Trees

Column Height Recurrence per Level

- $d_{k+1} = \text{floor}\left(\frac{3}{2} \times d_k\right)$
- Initial $d < \text{number of operands}$
- EX: Number of operands = 7
- Start from the largest valid 2 and reduce to 2
- $d=6 \rightarrow d=4 \rightarrow d=3 \rightarrow d=2$ Finish
- Dadda tree uses fewer reduction stages than Wallace
- **Reductions are postponed as much as possible without increasing delay**
- *This minimizes the number of adders while preserving optimal delay.*



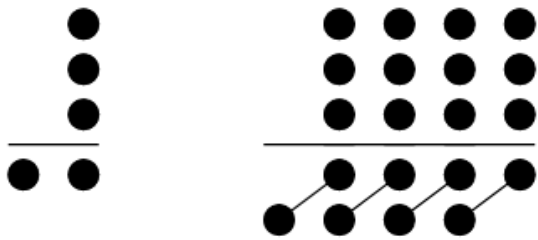
Multioperand Addition

Parallel Counters

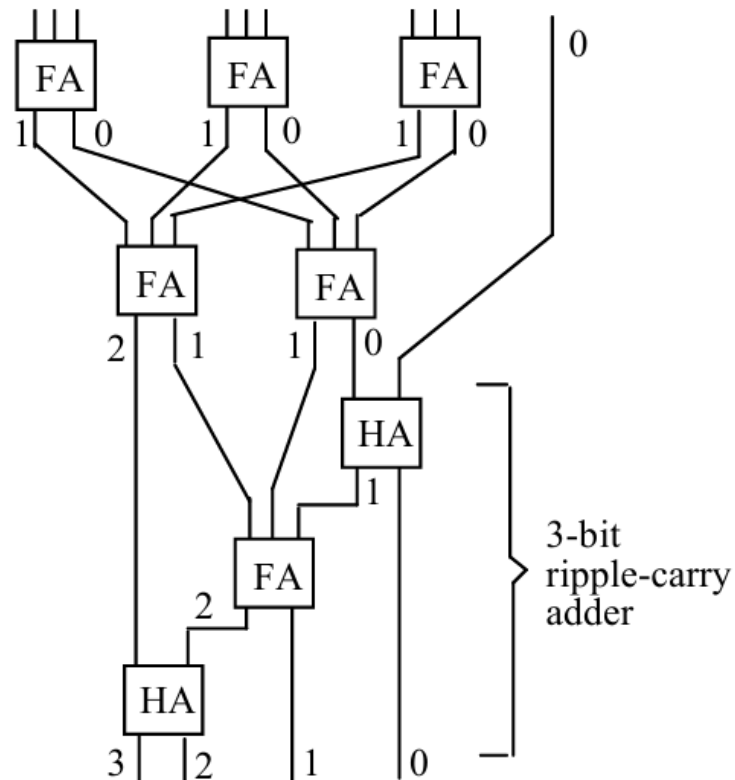
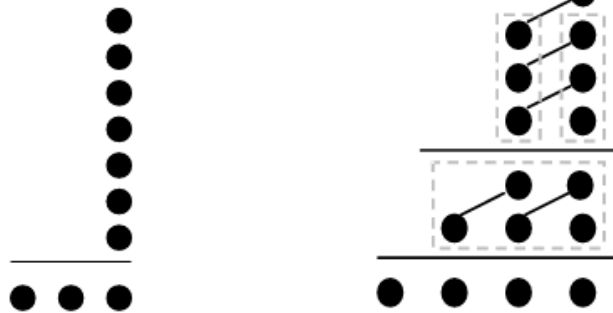
Parallel Counters and Compressors

- Circuits reducing n bits to their $\log_2(n + 1) - \text{bit sum} = (n; \log_2(n + 1))$

1-bit full-adder = (3; 2)-counter



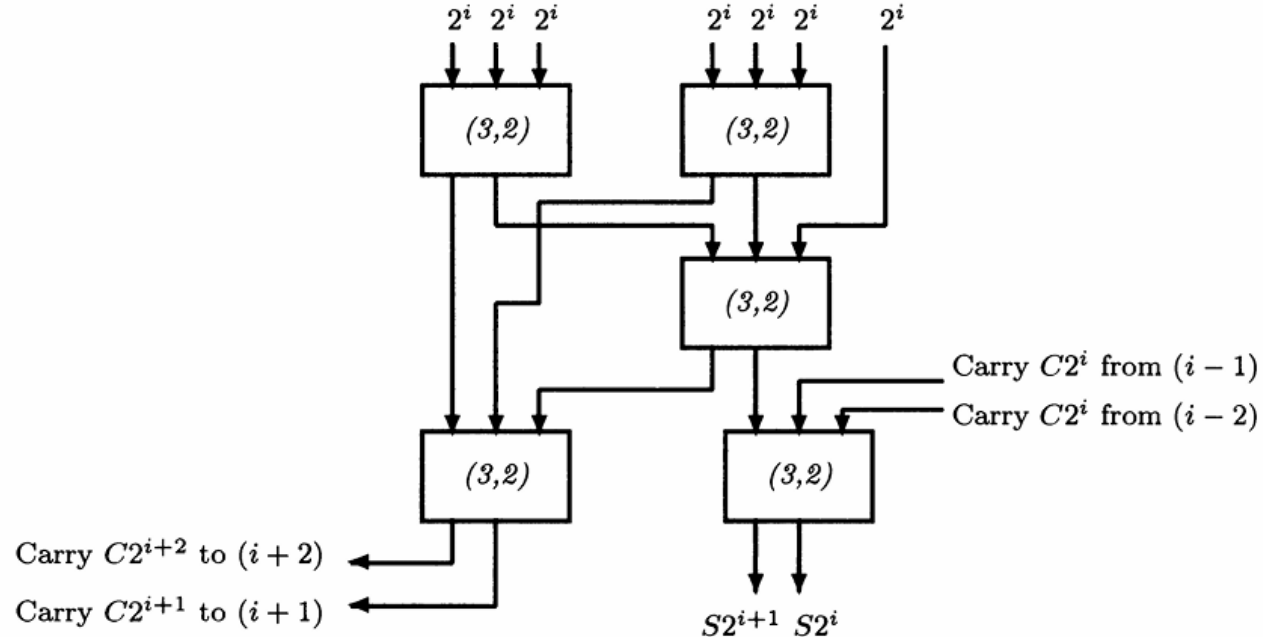
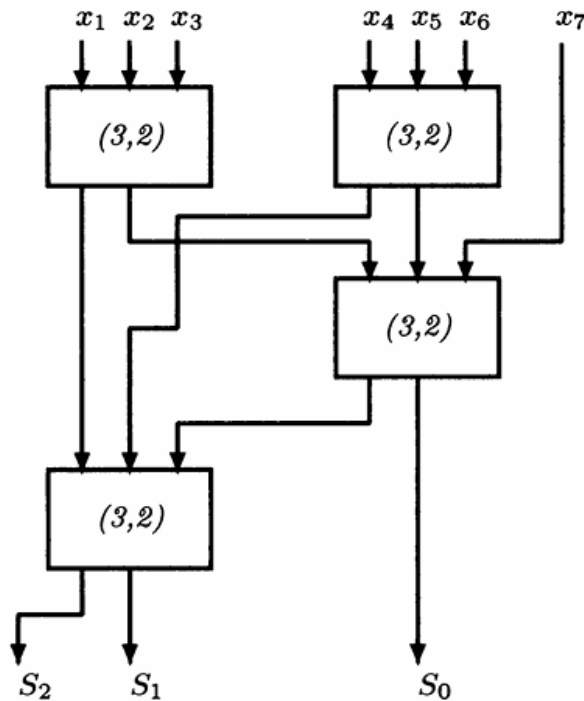
Circuit reducing 7 bits to their 3-bit sum = (7; 3)-counter



Parallel Counters

A 7:3 Compressor using 3:2 counters

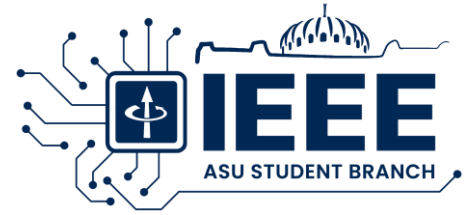
- Circuits reducing n bits to their $\log_2(n + 1)$ - bit sum = $(n; \log_2(n + 1))$



A 7:2 Compressor with two incoming carries and two outgoing carries, in bit position i .

Thank You

References



- B. Parhami, *Computer Arithmetic: Algorithms and Hardware Designs*, 2nd ed., Oxford University Press, New York/Oxford, 2010.
- B. Parhami, *ECE 252B Course Slides*, Chapters 5–8, University of California, Santa Barbara.
- H. A. Omran, *Digital IC Design – Lecture 26: Adders*, Integrated Circuits Laboratory (ICL), Electronics and Communications Engineering Dept., Faculty of Engineering, Ain Shams University. (Based mainly on N. Weste and D. Harris, *CMOS VLSI Design*, 4th ed.)
- A. Abdelsalam, *Computer Arithmetic – Adders Session*, STMicroelectronics Training Slides.
- I. Koren, *Computer Arithmetic Algorithms*, 2nd ed., A. K. Peters, Natick, MA, 2002, ISBN: 978-1568811604.
- Karim Abbas, *Handbook of Digital CMOS Technology, Circuits, and Systems*, Springer Nature Switzerland AG, 2020, ISBN 978-3-030-37194-4.